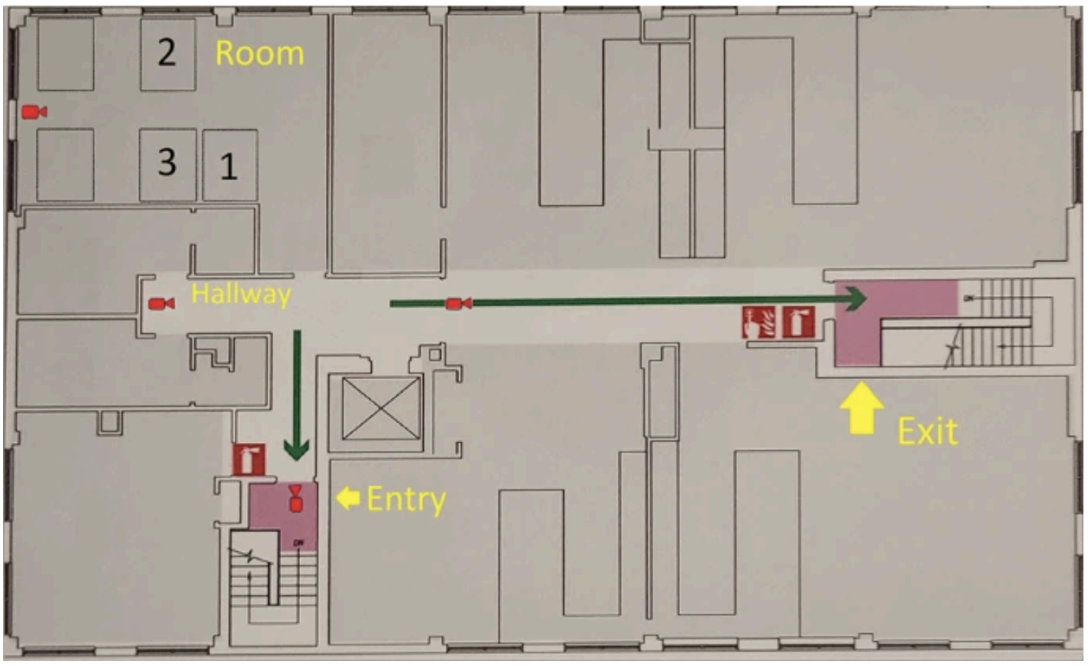
U24AI006

ASSIGNMENT -04

Q1 Use the city map given in Q1 of assignment 1 and implement the best first search algorithm to reach your destination and compare your results in terms of the number of possible paths explored.

Q2 Consider a building floor plan as given in figure. Design a robot guided evacuation planning using the best first search algorithm. Consider the following constraints:

- Allowed movements up, down, left and right
- Walls and rooms block movement
- Evacuation planning covers the route from entry to exit
- Design an evaluation cost function with proper justification for your algorithm



CODE:

```
cities = [
   "Syracuse", "Buffalo", "Pittsburgh", "Cleveland", "Columbus",
   "Detroit", "Chicago", "Indianapolis", "Boston", "New York",
   "Philadelphia", "Providence", "Baltimore", "Portland"
]

INF = 9999
n = 14
adj_matrix = [[INF] * n for _ in range(n)]


def connect(city1, city2, miles):
   idx1 = cities.index(city1)
   idx2 = cities.index(city2)
```

```
    adj_matrix[idx1][idx2] = miles
    adj_matrix[idx2][idx1] = miles

connect("Syracuse", "Buffalo", 150)
connect("Syracuse", "Boston", 312)
connect("Syracuse", "New York", 254)
connect("Syracuse", "Philadelphia", 253)
connect("Buffalo", "Detroit", 256)
connect("Buffalo", "Cleveland", 189)
connect("Buffalo", "Pittsburgh", 215)
connect("Boston", "New York", 215)
connect("Boston", "Providence", 50)
connect("Boston", "Portland", 107)
connect("New York", "Philadelphia", 97)
connect("New York", "Providence", 181)
connect("Philadelphia", "Baltimore", 101)
connect("Philadelphia", "Pittsburgh", 305)
connect("Baltimore", "Pittsburgh", 247)
connect("Pittsburgh", "Cleveland", 134)
connect("Pittsburgh", "Columbus", 185)
connect("Cleveland", "Detroit", 169)
connect("Cleveland", "Columbus", 144)
connect("Cleveland", "Chicago", 345)
connect("Detroit", "Chicago", 283)
connect("Columbus", "Indianapolis", 176)
connect("Indianapolis", "Chicago", 182)


class Node:
    def __init__(self, state_index, parent, path_cost):
        self.STATE = state_index
        self.PARENT = parent
        self.PATH_COST = path_cost

def EXPAND(node_u):
    child_nodes = []
    u_index = node_u.STATE


    for v_index in range(n):
        distance = adj_matrix[u_index][v_index]


        if distance != INF:
```

```
            new_cost = node_u.PATH_COST + distance
            child = Node(v_index, node_u, new_cost)
            child_nodes.append(child)

    return child_nodes

def BEST_FIRST_SEARCH(start_city_name, goal_city_name):
    start_index = cities.index(start_city_name)
    goal_index = cities.index(goal_city_name)


    start_node = Node(start_index, None, 0)

    frontier = [start_node]

    reached = {start_index: start_node}

    nodes_explored = 0

    print(f"Searching path from {start_city_name} to {goal_city_name}...")

    while len(frontier) > 0:

        frontier.sort(key=lambda x: x.PATH_COST)

        node = frontier.pop(0)
        nodes_explored += 1

        current_city = cities[node.STATE]

        if node.STATE == goal_index:
            return node, nodes_explored


        for child in EXPAND(node):
            s = child.STATE

            if (s not in reached) or (child.PATH_COST < reached[s].PATH_COST):


                reached[s] = child

                frontier.append(child)

    return None, nodes_explored
```

```python
solution_node, count = BEST_FIRST_SEARCH("Syracuse", "Chicago")

print("-" * 30)
if solution_node:
    path = []
    current = solution_node
    while current is not None:
        path.append(cities[current.STATE])
        current = current.PARENT
    path.reverse()

    print("Success!")
    print("Path taken:", " -> ".join(path))
    print("Total Miles:", solution_node.PATH_COST)
    print("Total nodes explored (popped):", count)
else:
    print("Failure: No path found.")
```

OUTPUT:

```
Searching path from Syracuse to Chicago...
------------------------------
Success!
Path taken: Syracuse -> Buffalo -> Cleveland -> Chicago
Total Miles: 684
Total nodes explored (popped): 15
```

CODE (2):

```
grid = [
    [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
    [1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1],
    [1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1],
    [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
    [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    [1, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1, 1, 1, 1, 1],
    [1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1],
    [1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1],
    [1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
    [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
]

rows = 10
cols = 20

START = (8, 4)
GOAL = (4, 18)

class Node:
    def __init__(self, state, parent=None, action=None, path_cost=0):
        self.state = state
        self.parent = parent
        self.action = action
        self.path_cost = path_cost

class PriorityQueue:
    def __init__(self, f):
        self.data = []
        self.f = f

    def add(self, node):
        self.data.append(node)
        self.data.sort(key=self.f)

    def pop(self):
        return self.data.pop(0)

    def top(self):
        return self.data[0]

    def is_empty(self):
```

```
        return len(self.data) == 0

class Problem:
    def __init__(self, initial, goal, grid):
        self.initial = initial
        self.goal = goal
        self.grid = grid

    def is_goal(self, state):
        return state == self.goal

    def ACTIONS(self, state):
        r, c = state
        actions = []
        moves = [
            ("Down",  (1, 0)),
            ("Up",    (-1, 0)),
            ("Right", (0, 1)),
            ("Left",  (0, -1))
        ]

        for name, (dr, dc) in moves:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols:
                if self.grid[nr][nc] == 0:
                    actions.append(name)
        return actions

    def RESULT(self, state, action):
        r, c = state
        if action == "Down":  return (r + 1, c)
        if action == "Up":    return (r - 1, c)
        if action == "Right": return (r, c + 1)
        if action == "Left":  return (r, c - 1)
        return state

    def ACTION_COST(self, s, action, s_prime):
        return 1

def heuristic(node):
    r1, c1 = node.state
    r2, c2 = GOAL
    return abs(r1 - r2) + abs(c1 - c2)

def f(node):
```

```
    return heuristic(node)

def EXPAND(problem, node):
    s = node.state
    for action in problem.ACTIONS(s):
        s_prime = problem.RESULT(s, action)
        cost = node.path_cost + problem.ACTION_COST(s, action, s_prime)
        yield Node(state=s_prime, parent=node, action=action, path_cost=cost)

def BEST_FIRST_SEARCH(problem, f):
    node = Node(problem.initial)
    frontier = PriorityQueue(f)
    frontier.add(node)

    reached = [[None for _ in range(cols)] for _ in range(rows)]
    reached[node.state[0]][node.state[1]] = node

    explored = 0

    while not frontier.is_empty():
        node = frontier.pop()
        explored += 1

        if problem.is_goal(node.state):
            return node, explored

        for child in EXPAND(problem, node):
            r, c = child.state
            if reached[r][c] is None or child.path_cost < reached[r][c].path_cost:
                reached[r][c] = child
                frontier.add(child)

    return None, explored

def get_path(node):
    path = []
    while node:
        path.append(node.state)
        node = node.parent
    return path[::-1]

def print_grid_with_path(grid, path):
    print("\nEvaluated Path on Grid:")
    path_set = set(path)
```

```
    print("   ", end="")
    for c in range(cols): print(f"{c%10}", end=" ")
    print()

    for r in range(rows):
        print(f"{r:<2} ", end="")
        for c in range(cols):
            if (r, c) == START:
                print("S", end=" ")
            elif (r, c) == GOAL:
                print("E", end=" ")
            elif (r, c) in path_set:
                print(".", end=" ")
            elif grid[r][c] == 1:
                print("#", end=" ")
            else:
                print(" ", end=" ")
        print()

if __name__ == "__main__":
    problem = Problem(START, GOAL, grid)

    print("Starting Best First Search (Greedy)...")
    print(f"Start: {START}, Goal: {GOAL}")

    solution, explored = BEST_FIRST_SEARCH(problem, f)

    if solution:
        path = get_path(solution)
        print("\nGoal Found!")
        print(f"Path Length: {len(path)}")
        print(f"Total Cost: {solution.path_cost}")
        print(f"Nodes Explored: {explored}")

        print_grid_with_path(grid, path)

        print("\nPath Steps:")
        curr = solution
        steps = []
        while curr.parent:
            steps.append(f"{curr.action} -> {curr.state}")
            curr = curr.parent
        for s in reversed(steps):
            print(s)
```

```
        print("\nEvaluation Cost Function Justification:")
        print("Function: Manhattan Distance f(n) = |x1 - x2| + |y1 - y2|")
        print("Reason: Since robots move in 4 cardinal directions,")
        print("Manhattan distance perfectly estimates the minimum number of steps to
reach the goal")
        print("ignoring obstacles. It is simpler to calculate than Euclidean
distance and")
        print("more accurate for grid-based movement.")


    else:
        print("No path found!")
```

OUTPUT:

```
Starting Best First Search (Greedy)...
Start: (8, 4), Goal: (4, 18)

Goal Found!
Path Length: 21
Total Cost: 20
Nodes Explored: 30

Evaluated Path on Grid:
   0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9
0  # # # # # # # # # # # # # # # # # # # #
1  #     #     # # # #     #     #       #
2  #     #     # # # #     #     #       #
3  # # # # # # # # # #     # # # # # # # #
4  #     . . . . . . . . . . . . . . . E
5  # # # . # # # #     # #       # # # # #
6  #     . .     #     # #       #       #
7  # # # # .     #     # #       #       #
8  #     # S     # # # # # # # # # # # # #
9  # # # # # # # # # # # # # # # # # # # #

Path Steps:
Up -> (7, 4)
Up -> (6, 4)
Left -> (6, 3)
Up -> (5, 3)
Up -> (4, 3)
Right -> (4, 4)
Right -> (4, 5)
Right -> (4, 6)
Right -> (4, 7)
Right -> (4, 8)
Right -> (4, 9)
Right -> (4, 10)
Right -> (4, 11)
Right -> (4, 12)
Right -> (4, 13)
Right -> (4, 14)
Right -> (4, 15)
Right -> (4, 16)
Right -> (4, 17)
Right -> (4, 18)

Evaluation Cost Function Justification:
Function: Manhattan Distance f(n) = |x1 - x2| + |y1 - y2|
Reason: Since robots move in 4 cardinal directions,
Manhattan distance perfectly estimates the minimum number of steps to reach the goal
ignoring obstacles. It is simpler to calculate than Euclidean distance and
more accurate for grid-based movement.
```